

Práctica 2

Trabajando con el Turtlebot 3 en simulación

Abel Gandía Ruiz

<https://github.com/danisty/tar>

Índice

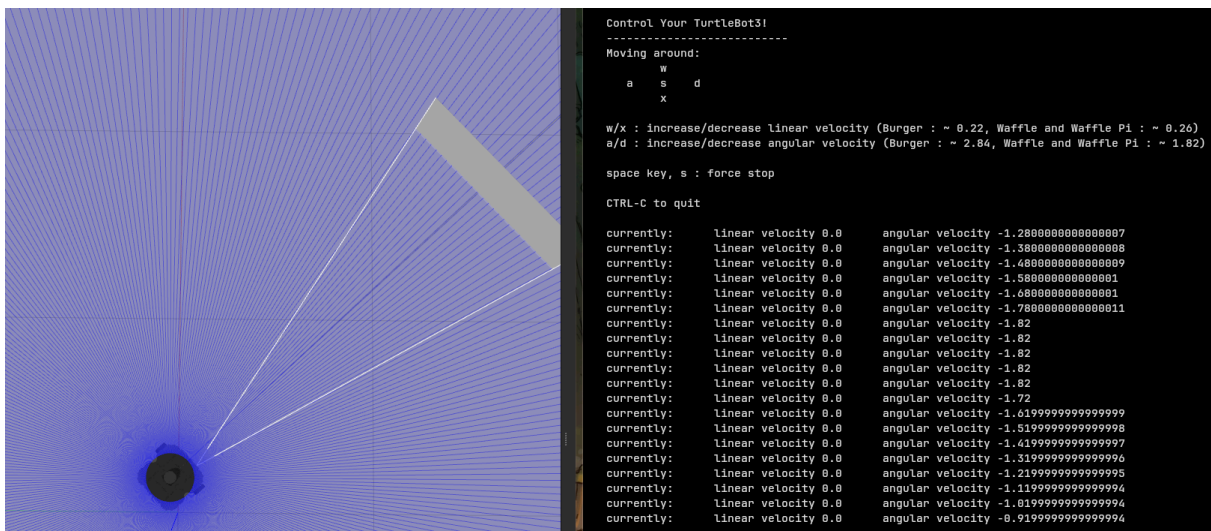
Parte 1: Primeros Pasos con el Turtlebot y Odometría	3
Preguntas teóricas	3
Ejercicios	6

Parte 1: Primeros Pasos con el Turtlebot y Odometría

Preguntas teóricas

Para esta primera parte se lanza el Turtlebot 3 **Waffle** en **Gazebo** y se teleopera con el paquete **turtlebot3_teleop**. Esto nos permite estudiar los mecanismos de comunicación que emplea ROS 2 para controlar al robot.

```
export TURTLEBOT3_MODEL=waffle
ros2 run turtlebot3_teleop teleop_keyboard
```



PREGUNTA 1

¿Cuál es el topic en el cual se debe publicar la información para que el robot se mueva?

El topic sobre el que se publica para mover al robot es **/cmd_vel**. Se puede comprobar inspeccionando el nodo de teleoperación:

```
ros2 node info /teleop_keyboard
```

```
/teleop_keyboard
Subscribers:

Publishers:
  /cmd_vel: geometry_msgs/msg/Twist
  ...
Service Servers:
  ...
  ...
```

PREGUNTA 2

¿Cuál es el tipo de mensaje que se publica?

El tipo de mensaje publicado en `/cmd_vel` es `geometry_msgs/msg/Twist`. Se puede verificar con:

```
ros2 topic info /cmd_vel
```

```
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1
```

El mensaje `Twist` contiene dos vectores de tres componentes cada uno:

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
```

Para el Turtlebot 3, al tratarse de un robot diferencial que se mueve sobre un plano, únicamente se utilizan `linear.x` (velocidad de avance) y `angular.z` (velocidad de giro).

PREGUNTA 3

¿Qué tipo de movimientos puede realizar el robot? ¿Cuáles son los ejes de movimientos positivos?

El Turtlebot 3 es un robot de **tracción diferencial** con dos ruedas motrices. Esto le permite realizar dos tipos de movimiento:

1. **Movimiento lineal** a lo largo del eje X del robot: `linear.x > 0` avanza hacia adelante, `linear.x < 0` retrocede.
2. **Movimiento rotacional** alrededor del eje Z (vertical): `angular.z > 0` gira en sentido **antihorario** (izquierda), `angular.z < 0` gira en sentido **horario** (derecha).

No puede desplazarse lateralmente (no tiene movimiento en el eje Y), por tanto, si le mandamos un giro de -20° , el robot girará hacia la **derecha**.

PREGUNTA 4

¿El robot gira sobre su eje siempre? ¿Qué comando de odometría se le está mandando a los motores para que sea así?

Sí, el Turtlebot 3 gira sobre su propio eje cuando se le envía únicamente velocidad angular (`angular.z != 0`) con velocidad lineal nula (`linear.x = 0`). En este caso, ambas ruedas giran a la misma velocidad pero en **sentidos opuestos**, haciendo que el punto medio del eje de las ruedas se mantenga fijo.

PREGUNTA 5

¿Cuál es la unidad de magnitud de las velocidades lineales y angulares?

Las unidades en ROS 2 siguen el sistema internacional (SI):

- › **Velocidad lineal:** metros por segundo (m/s)
- › **Velocidad angular:** radianes por segundo (rad/s)

Ejercicios

EJERCICIO 1

Nodo `movimiento.py` con 4 modos de movimiento y nodo `dibuja_mov.py` para visualización de trayectorias

Se ha creado el paquete `p3_pkg` con las dependencias necesarias (`rclpy`, `geometry_msgs`, `nav_msgs`). Dentro de este paquete se implementan dos nodos: `movimiento.py` para ejecutar las trayectorias y `dibuja_mov.py` para registrarlas y dibujarlas.

Cada modo de movimiento define una cola de pasos `('forward', distancia)` o `('rotate', grados)`. El callback de odometría (`/odom`) compara la posición/ángulo actual con el origen del paso y, cuando se alcanza la tolerancia, pasa al siguiente paso.

El nodo soporta dos modos de control, seleccionables con el flag `--pid`:

- › **Sin PID (por defecto):** velocidad constante; útil para observar el *drift* odométrico en los ejercicios 1-2.
- › **Con PID (`--pid`):** controlador PID sobre la rotación y corrección de *heading* en avance; detallado en el ejercicio 4.

MODO 0: MOVIMIENTO LINEAL DE 2 METROS

```
if movement_type == 0:  
    self.steps.append(('forward', 2.0))
```

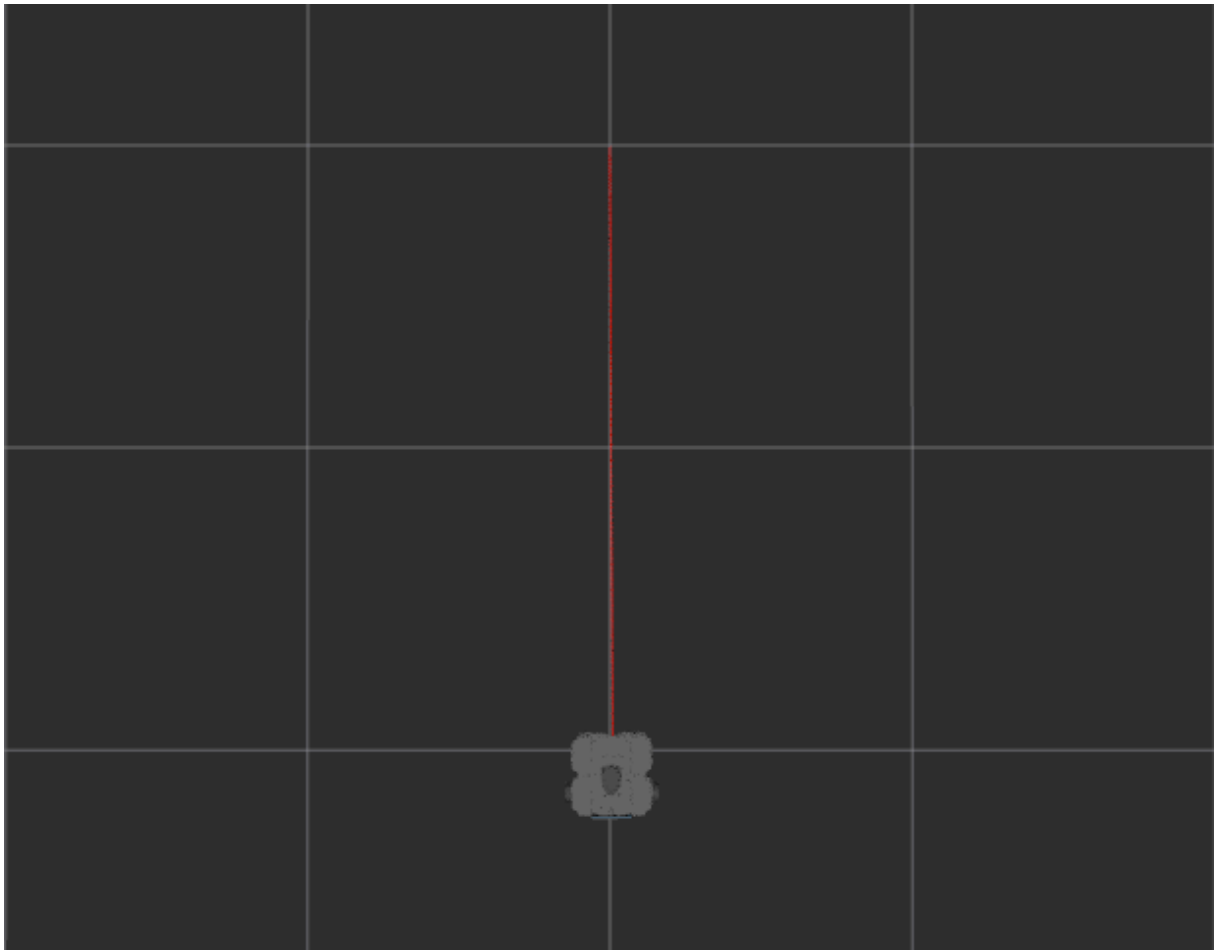
El robot avanza 2 metros en línea recta midiendo la distancia recorrida a través de la odometría.

PREGUNTA 6

¿Cómo puedo comprobar que el robot haya avanzado esta distancia?

Se puede comprobar suscribiéndose al topic `/odom` de tipo `nav_msgs/msg/Odometry`. Este topic publica la posición estimada del robot. Comparando `pose.pose.position.x` antes y después del movimiento se puede calcular la distancia recorrida con:

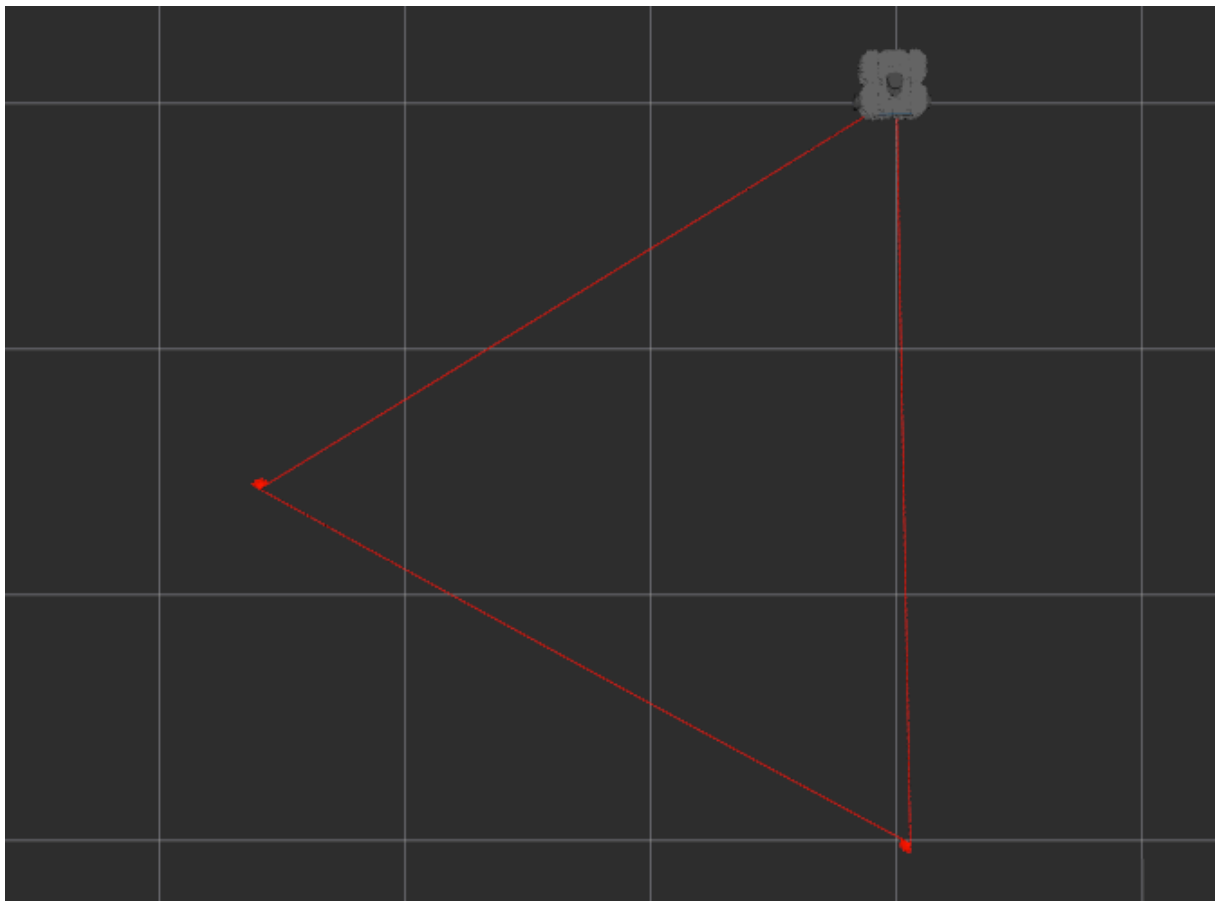
```
ros2 topic echo /odom
```



MODO 1: TRIÁNGULO EQUILÁTERO DE 3M DE LADO

```
elif movement_type == 1:  
    for _ in range(3):  
        self.steps.append(('forward', 3))  
        self.steps.append(('rotate', -120))
```

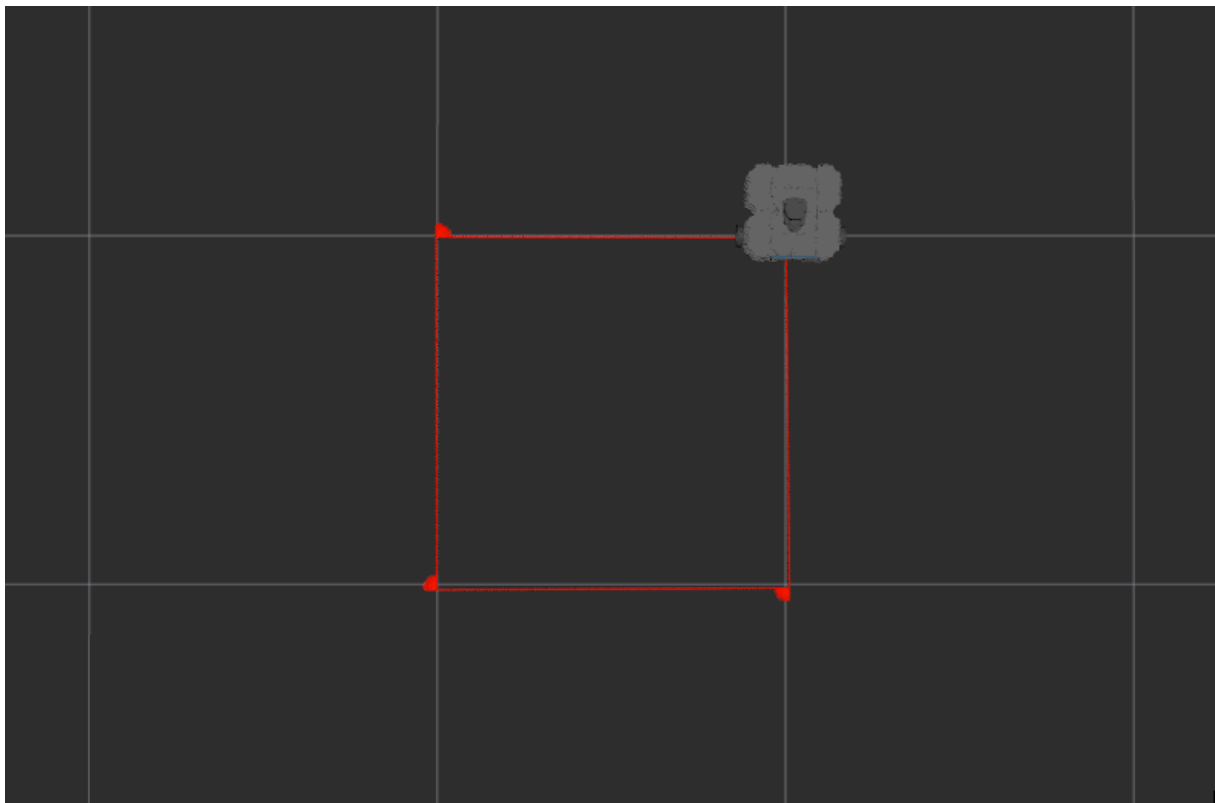
El robot avanza 3 metros y gira -120° (horario) tres veces. El ángulo exterior de un triángulo equilátero es $\frac{360^\circ}{3} = 120^\circ$, con signo negativo para girar a la derecha.



MODO 2: CUADRADO DE 1M DE LADO

```
elif movement_type == 2:  
    for _ in range(4):  
        self.steps.append(('forward', 1))  
        self.steps.append(('rotate', -90))
```

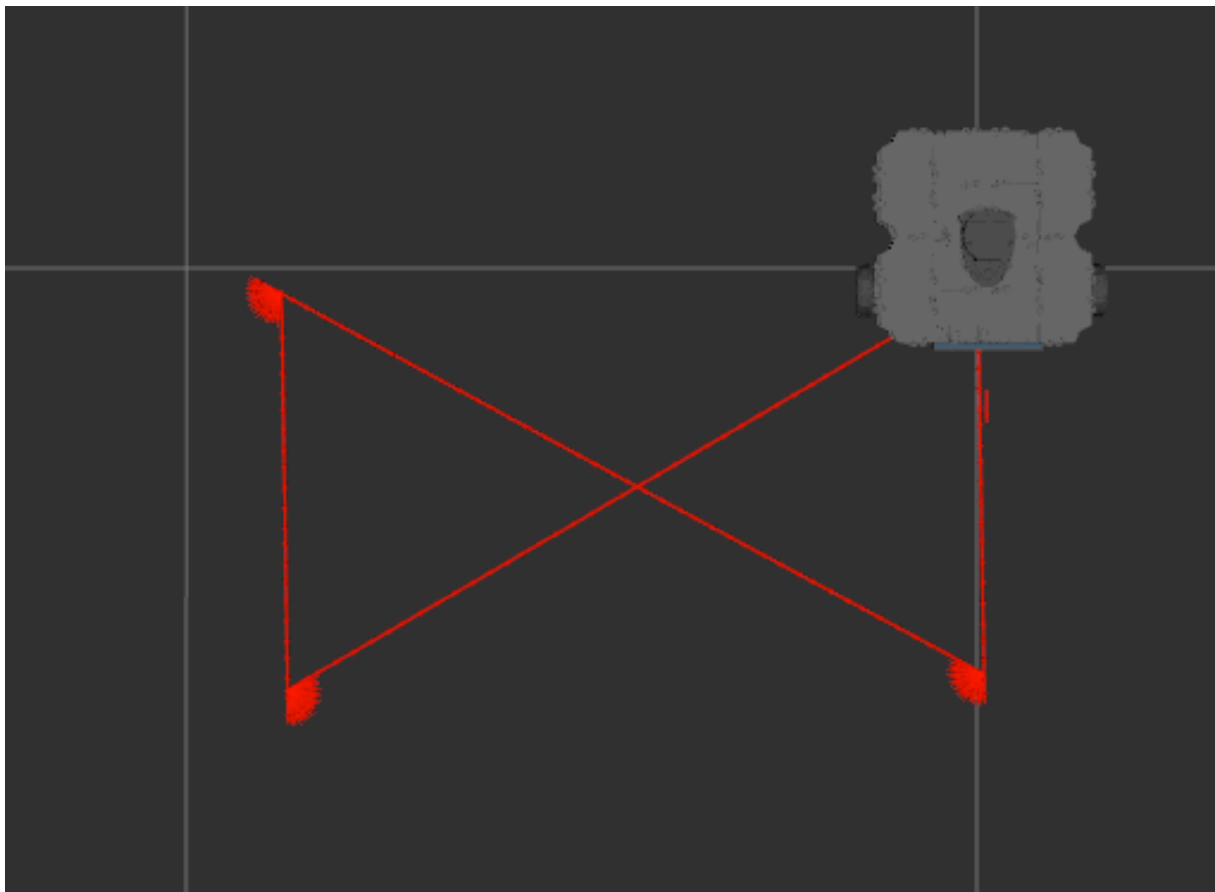
El robot avanza 1 metro y gira -90° cuatro veces, dibujando un cuadrado.



MODO 3: FIGURA DE INFINITO (BOWTIE)

```
elif movement_type == 3:
    self.steps.append(('forward', 0.5))
    self.steps.append(('rotate', -120))
    self.steps.append(('forward', 1))
    self.steps.append(('rotate', 120))
    self.steps.append(('forward', 0.5))
    self.steps.append(('rotate', 120))
    self.steps.append(('forward', 1))
    self.steps.append(('rotate', -120))
```

La figura de infinito se construye como dos triángulos simétricos (un *bowtie*). El robot avanza 0.5m, gira para trazar el primer triángulo, vuelve al centro y traza el segundo triángulo en espejo.



EJECUCIÓN

Terminal 1: Lanzar Gazebo con el Turtlebot 3

```
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

Terminal 2: Lanzar RViz2 para visualizar la trayectoria en tiempo real

```
rviz2
```

Se configura el *display* **Odometry** sobre el topic `/odom` con **Keep** elevado (> 1000) para acumular la traza, y se establece el *Fixed Frame* a `odom`. Opcionalmente se añade el *display* **RobotModel** para visualizar el robot.

Terminal 3: Ejecutar el movimiento deseado

```
ros2 run p3_pkg movimiento <modo> # 0, 1, 2 o 3
```

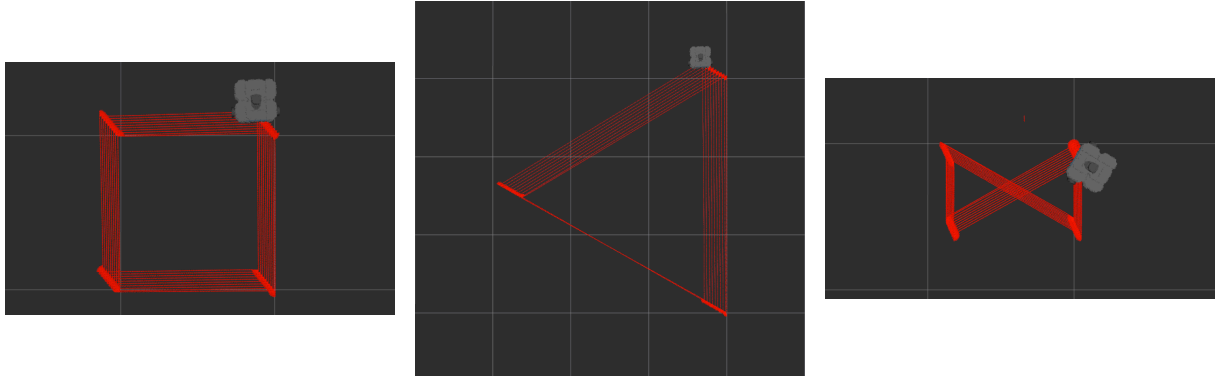
EJERCICIO 2

Ejecutar 10 veces el cuadrado, triángulo e infinito y analizar la acumulación de error

Se ejecutan los modos 1, 2 y 3 del nodo `movimiento.py` diez veces consecutivas sin resetear la simulación entre ejecuciones. RViz2 acumula la traza de odometría, permitiendo visualizar el *drift* progresivo.

RESULTADOS

Al repetir cada trayectoria 10 veces, se observa que el robot **no vuelve exactamente a la posición original** en cada iteración. El error se acumula progresivamente, haciendo que la trayectoria se desplace respecto al punto de inicio.



PREGUNTA

¿El robot acaba en la posición original? ¿A qué puede deberse esto?

No, el robot **no vuelve exactamente** a la posición inicial tras cada repetición. El error se acumula con cada iteración, desplazando progresivamente el punto de retorno.

Este problema se conoce como **drift odométrico** y es un problema fundamental de la odometría basada en ruedas (*dead reckoning*). Es la razón por la que los robots reales emplean sensores adicionales (LiDAR, cámaras, IMU) para corregir su estimación de posición.

EJERCICIO 3

Nodo `aparcamiento.py` para estacionar el Turtlebot 3 en un espacio delimitado

Se ha implementado el nodo `aparcamiento.py` dentro del paquete `p3_pkg`.

ENTORNO DE SIMULACIÓN

Para lanzar el entorno de aparcamiento:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch p3_pkg parking_tb3.launch.py
```

El mundo `parking.world` define un espacio de aparcamiento con las siguientes coordenadas:

- › Pared izquierda: $x \approx 1.0$
- › Pared derecha: $x \approx 2.0$
- › Pared trasera: $y \approx -1.95$
- › Entrada abierta: $y \approx -1.0$
- › Centro del hueco: $x \approx 1.5$

El robot arranca en $(0, 0)$ mirando hacia $+X$.

ESTRATEGIA DE APARCAMIENTO

Se realiza una maniobra de **aparcamiento en marcha atrás** (*reverse bay parking*) en tres pasos:

1. **Avanzar 1.5m:** el robot se desplaza en línea recta hacia $+X$ hasta alinearse con el centro del hueco ($x \approx 1.5$).
2. **Girar 90°:** rotación antihoraria para orientar el robot hacia $+Y$, quedando de espaldas a la entrada del espacio.
3. **Marcha atrás 1.4m:** el robot entra marcha atrás en el hueco hasta quedar centrado.

```
def run(self):
    self.steps.append(('forward', 1.5))
    self.steps.append(('rotate', 90))
    self.steps.append(('reverse', 1.4))
```

CÓDIGO

```
class AparcamientoNode(Node):
    def __init__(self):
        super().__init__('aparcamiento_node')
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        self.create_subscription(Odometry, 'odom', self._odom_cb, 10)
        self.steps = deque()
        # ... (estado interno: posición, ángulo, acción actual)

    def _forward_ctrl(self):
        dist = math.hypot(self.x - self.origin_x, self.y - self.origin_y)
        remaining = self.target - dist
        if remaining < LINEAR_TOLERANCE:
            self._stop()
            self._next()
        else:
```

```

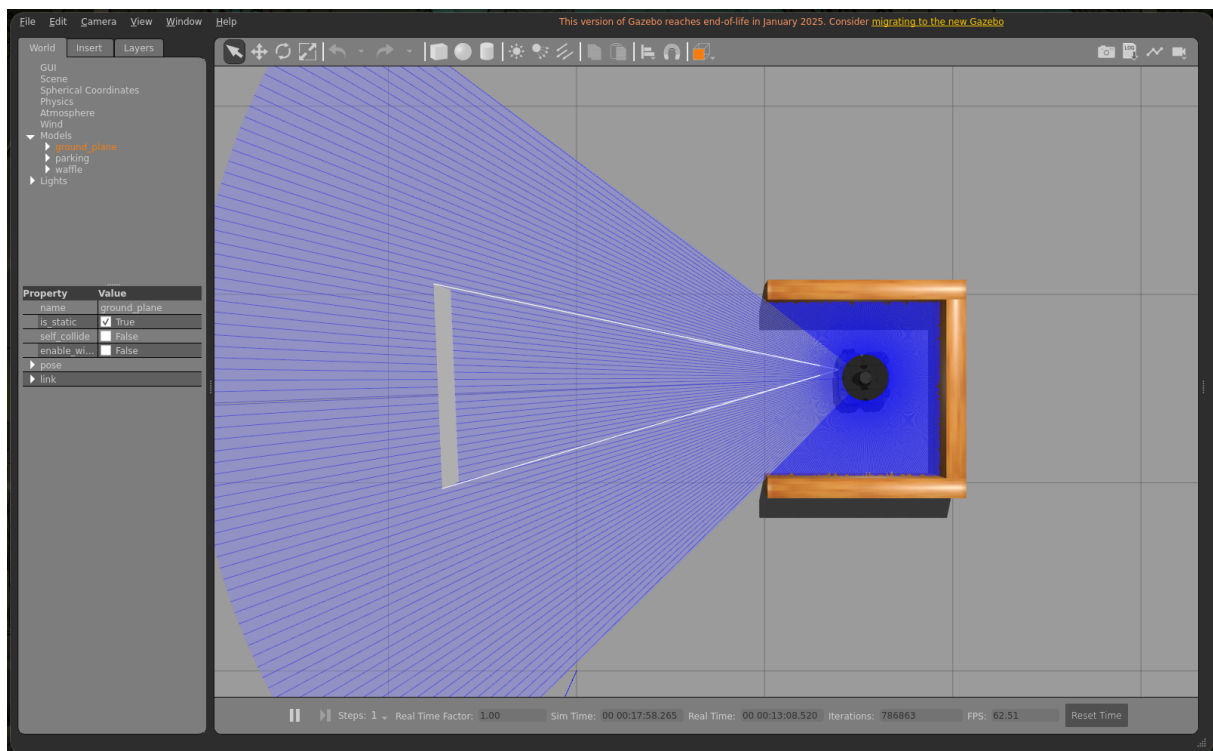
speed = min(SPEED, max(0.05, remaining * 1.5))
if self.action == 'reverse':
    speed = -speed
msg = Twist()
msg.linear.x = speed
self.pub.publish(msg)

def _rotate_ctrl(self):
    angle = math.atan2(
        math.sin(self.theta - self.origin_theta),
        math.cos(self.theta - self.origin_theta))
    remaining = self.target - angle
    if abs(remaining) < ANGULAR_TOLERANCE:
        self._stop()
        self._next()
    else:
        w = min(ANGLE_SPEED, max(0.05, abs(remaining) * 1.5))
        msg = Twist()
        msg.angular.z = w if remaining > 0 else -w
        self.pub.publish(msg)

```

EJECUCIÓN

```
ros2 run p3_pkg aparcamiento
```



EJERCICIO 4

Corrección de errores de odometría mediante un controlador PID

Como se ha observado en los Ejercicios 1 y 2, el robot no ejecuta las trayectorias de forma perfecta. Para corregir estos errores se ha implementado un **controlador PID** (*Proportional-Integral-Derivative*) dentro del nodo `movimiento.py`, activable mediante el flag `--pid`.

```
ros2 run p3_pkg movimiento 2 --pid # cuadrado con PID
ros2 run p3_pkg movimiento 2 10 --pid # cuadrado 10 veces con PID
```

Se implementan dos controladores PID independientes:

1. PID de rotación: controla los giros puros (`angular.z`):

```
KP_ROT = 0.5 # Proporcional: corrección suave
KI_ROT = 0.01 # Integral: compensa errores constantes
KD_ROT = 0.05 # Derivativo: amortigua oscilaciones
```

2. PID de corrección de heading: mantiene la línea recta durante el avance:

```
KP_HDG = 1.0 # Proporcional: corrección moderada
KI_HDG = 0.0 # Integral: desactivado (no hay sesgo constante)
KD_HDG = 0.1 # Derivativo: suaviza las correcciones
```

Los valores se han ajustado a partir de implementaciones de referencia para el Turtlebot 3.

Al iniciar cada avance, se memoriza el *heading* actual como objetivo (`target_theta`). Si durante el avance el robot se desvía, el PID aplica una corrección en `angular.z` para reconducirlo.

IMPLEMENTACIÓN DEL PID

```
def _pid(self, error, kp, ki, kd):
    now = time.monotonic()
    dt = now - self._prev_time if self._prev_time else 0.0
    self._prev_time = now

    self._integral += error * dt
    self._integral = max(-1.0, min(1.0, self._integral)) # anti-windup

    derivative = (error - self._prev_error) / dt if dt > 0 else 0.0
    self._prev_error = error

    return kp * error + ki * self._integral + kd * derivative
```

El mecanismo de **anti-windup** limita la integral entre $[-1, 1]$ para evitar que se acumule un valor excesivo cuando el error persiste.

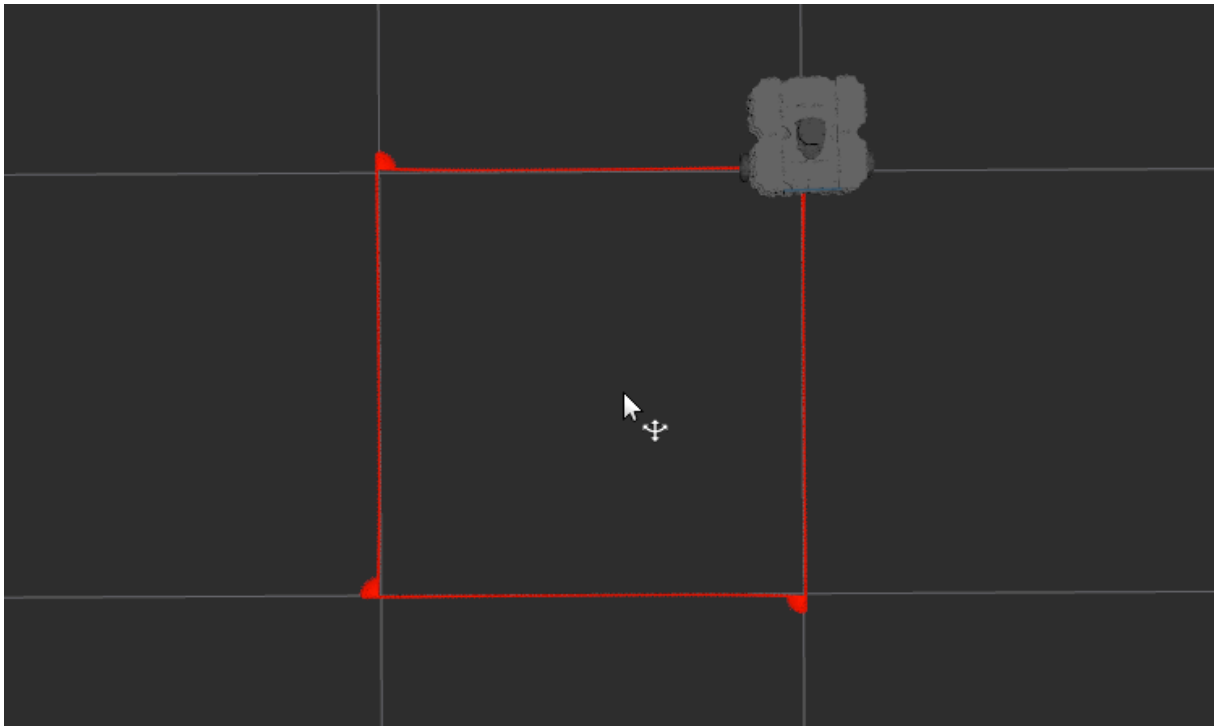
TOLERANCIA ANGULAR

Con PID activo, la tolerancia angular se reduce a 0.002 rad ($\approx 0.11^\circ$), frente a los 0.02 rad ($\approx 1.15^\circ$) del modo sin PID. Esto es posible porque el PID desacelera suavemente al aproximarse al ángulo objetivo, mientras que el control a lazo abierto necesita una tolerancia mayor para evitar sobrepasos.

RESULTADO

Con el controlador PID, las trayectorias más precisas:

- › Los giros se completan con un error inferior a 0.002 radianes ($\approx 0.11^\circ$)
- › Los avances mantienen el *heading* con correcciones en tiempo real
- › Las velocidades se modulan proporcionalmente a la distancia restante, evitando sobrepasos



EJERCICIO 5

Repetir los ejercicios con el Turtlebot 3 Burger y comparar resultados

Se repiten los ejercicios 1-4 empleando el modelo **Burger** en lugar del **Waffle** :

```
export TURTLEBOT3_MODEL=burger
```

Captura de pantalla

PREGUNTA

¿Observas diferencias en la precisión de los movimientos entre Burger y Waffle? ¿Cuál robot acumula más error? ¿En qué tipo de movimiento se nota más la diferencia?

PREGUNTA

¿Burger realiza giros más amplios o más cerrados que Waffle?

PREGUNTA

Al implementar controladores para corregir errores de trayectoria: ¿Los mismos parámetros PID funcionan igual para ambos robots?

PREGUNTA

Durante la tarea de aparcamiento: ¿Cuál robot realiza la maniobra con menos correcciones?

PREGUNTA

Inspecciona las propiedades URDF de ambos robots: ¿Cómo afectan las diferencias a la precisión y estabilidad?

Las propiedades URDF se encuentran en el paquete `turtlebot3_gazebo/urdf/`. Las principales diferencias son:

Propiedad	Burger	Waffle
Radio de rueda	~0.033 m	~0.033 m
Separación de ruedas	~0.160 m	~0.287 m
Masa total	~1.0 kg	~1.8 kg
Sensor LiDAR	LDS-01 (360°)	LDS-01 (360°)
Cámara	No	Sí (Intel RealSense)

TODO: Comentar diferencias